

Assembly DLL EXE in .NET Framework

In this article, I am going to discuss **Assembly DLL and EXE in .NET Framework** with Examples. Please read our previous article where we discussed [Managed and Unmanaged Code in .NET Framework](#). As part of this article, we are going to discuss the following pointers in detail.

1. [What is an Assembly in .NET?](#)
2. [Types of Assemblies of in .NET Framework.](#)
3. [Understanding DLL and EXE.](#)
4. [What is the difference between the DLL and the EXE in .NET Framework?](#)

What is an Assembly in .NET?

According to MSDN, Assemblies are the building block of .NET Framework applications. They form the fundamental unit of deployment. In simple words, we can say that Assembly is nothing but a precompiled .NET Code that can be run by CLR (Common Language Runtime).

Let us understand the above definition with an example. In order to understand this, let us create a simple console application with the name MyConsoleApp. Once you created the console application then please modify the Program class as shown below.

```
using System;
namespace MyConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("This is From Console App");
            Console.ReadKey();
        }
    }
}
```

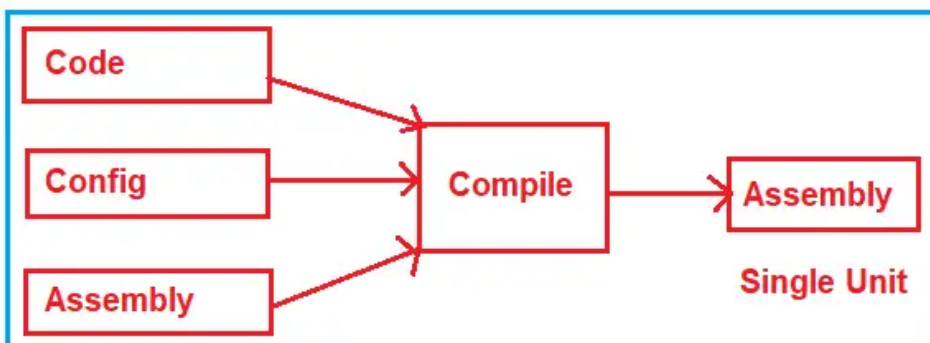
Now, if you right-click on your project and click on Open Folder in File Explorer, then you will find lots of things (Source code i.e. Program.cs class file, Configuration file i.e. App, Properties folder which contains AssemblyInfo.cs class file, etc.) as shown in the below image.

> This PC > LENOVO (D:) > MyConsoleApp > MyConsoleApp				
Name	Date modified	Type	Size	
bin	25-02-2020 20:46	File folder		
obj	25-02-2020 20:46	File folder		
Properties	25-02-2020 20:46	File folder		
App	25-02-2020 20:46	XML Configuration...	1 KB	
MyConsoleApp	25-02-2020 20:46	Visual C# Project fi...	3 KB	
Program.cs	25-02-2020 20:48	Visual C# Source F...	1 KB	

But when you build the application, then it will put the above whole thing into a single EXE as shown in the below image. You can find this EXE file under the **bin => Debug** folder. You can copy this single unit i.e. **MyConsoleApp.exe** and put it anywhere on your computer and from there you can run it.

> This PC > LENOVO (D:) > MyConsoleApp > MyConsoleApp > bin > Debug				
Name	Date modified	Type	Size	
MyConsoleApp	25-02-2020 20:55	Application	5 KB	
MyConsoleApp.exe	25-02-2020 20:46	XML Configuration...	1 KB	
MyConsoleApp.pdb	25-02-2020 20:55	Program Debug D...	12 KB	
MyConsoleApp.vshost	25-02-2020 20:47	Application	23 KB	
MyConsoleApp.vshost.exe	25-02-2020 20:46	XML Configuration...	1 KB	
MyConsoleApp.vshost.exe.manifest	19-03-2019 10:16	MANIFEST File	1 KB	

So, an assembly is nothing but a single unit of deployment or it is a precompiled chunk of code that can be executed by CLR. For better understanding please have a look at the following diagram.



Types of Assemblies of in .NET Framework:

In the .NET Framework, there are two types of assemblies. They are as follows:

1. **EXE (Executable)**
2. **DLL (Dynamic Link Library)**

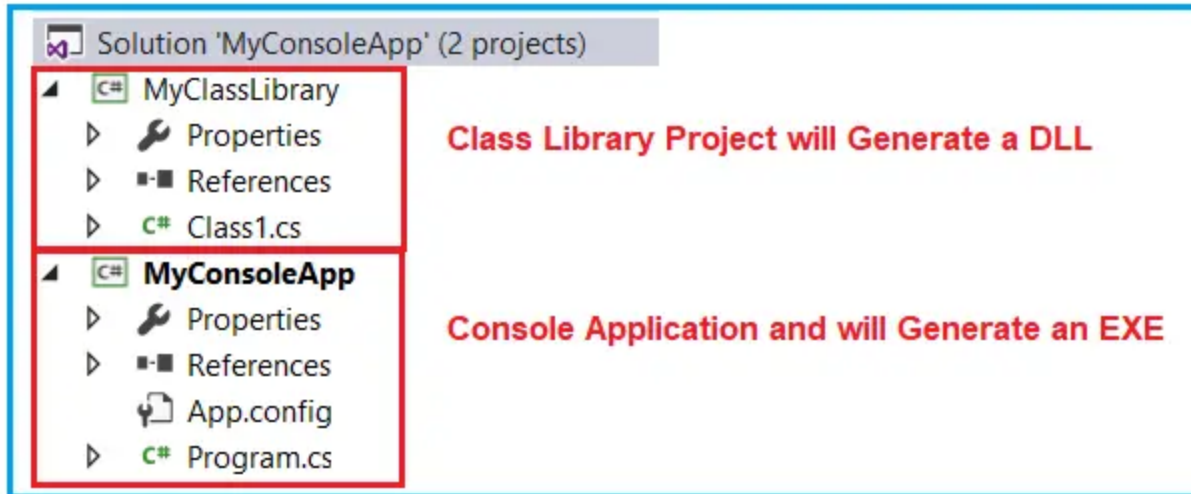
In .NET Framework when we compile a Console Application or a Windows Application, it generates EXE, whereas when we compile a Class Library Project or ASP.NET Web (MVC or Web API) application, then it generates DLL. In .NET framework, both EXE and DLL are called assemblies.

Understanding DLL and EXE in .NET Framework:

We already created one console application and we already see that it creates an EXE. Let us see an example of DLL. In order to create a DLL, let us add a class library project to the same solution with the name as **MyClassLibrary**. Once you created the class library project, it will by default create a class file with the name Class1. Let us modify Class1 as shown below.

```
namespace MyClassLibrary
{
    public class Class1
    {
        public string GetData()
        {
            return "This is from Class Library";
        }
    }
}
```

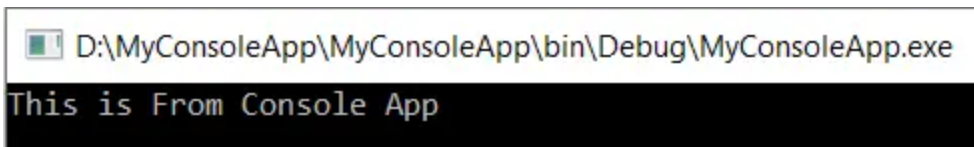
With this, now our solution contains two projects. One is a console application and the other one is a class library project as shown below.



Now, build the solution and you should get the respected assemblies as expected in their own **bin => Debug** folder. Now, the question that should come to your mind is what is the difference between DLL and EXE in .NET Framework?

What is the difference between the DLL and the EXE in .NET Framework?

The EXE is run in its own address space or in its own memory space. If you double-click on the MyConsoleApp.EXE file, then you will get the following output. Now, this program is running out of its own memory space.



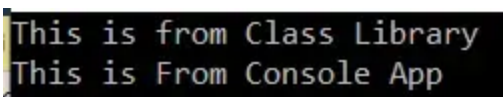
Without closing this window, again if you double-click on the MyConsoleApp.EXE file, again it will run and will display the same output. This is because now, both the EXE are running in their own memory space. The point that you need to remember is that EXE is an executable file and can run by itself as an application.

Coming to DLL, it cannot be run by itself like EXE. That means the **MyClassLibrary.dll** cannot be invoked or run by itself. It needs a consumer who is going to invoke it. So, a DLL is run inside other memory space. The other memory space can be a Console, Windows Applications, or Web Applications that should have their own memory space.

For example, you can invoke the DLL from a Console Application. We have a Console Application called MyConsoleApp and let's see how to invoke the **MyClassLibrary.dll** from this console application. In order to use the MyClassLibrary.dll inside the MyConsoleApp, first, you need to make a reference to that DLL. Once you add a reference to MyClassLibrary DLL, and then please modify the Program class of the Console Application as shown below.

```
using System;
using MyClassLibrary;
namespace MyConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using MyClassLibrary DLL
            Class1 obj = new Class1();
            Console.WriteLine(obj.GetData());
            Console.WriteLine("This is From Console App");
            Console.ReadKey();
        }
    }
}
```

Now, run the application and you should see the following output. Here, the MyClassLibrary DLL is run inside the MyConsoleApp address space.



So, in short, the difference between them is an EXE is an executable file and can run by itself as an application whereas DLL is usually consumed by an EXE or by another DLL and we cannot run or execute DLL directly. Even a Web Server like IIS can also invoke the DLL in the case of Web Applications.

Now, the question that should come to your mind why do we need DLLs as it is not invoked by themselves. The reason behind the DLL is reusability. Suppose you want some class, or logic, or something else in many applications, then simply put those classes, and logic inside a DLL, and refer to that DLL wherever it is required.

In the next article, I am going to discuss [App Domain in .NET Framework](#). Here, in this article, I try to explain **Assembly DLL EXE in .NET Framework** with Examples. I hope you enjoy this article and got a better idea of these three concepts.